- Very good work. 78%

- The report shows very deep understanding of the material well done.

DSD Final Report

Group 2:
Diederik Adriaan Vink (dav114)
Jesus Erlantz Garcia Condado (jeg114)

March 17, 2017

Contents

1 Task 6: Add Hardware Floating point Units 2
1.1 Setting up custom instructions . . . 2
1.2 Our hardware floating point unit . . . 2
1.2.1 Performance Results . . . 3
1.2.2 Cache Size Optimization . . . 3
1.2.3 Result Accuracy . . . 4
1.3 Extending the use of our hardware . . . 4

2 Task 7: Add Dedicated Hardware for f(x) 6
2.1 CORDIC Algorithm Implementation . . . 6
2.2 Implementing the full instruction . . . 7
2.3 Task 7 Results . . . 9
2.3.1 Latency Reduction Effects . . . 9
2.3.2 Cache Size Optimization . . . 10
2.3.3 Result Accuracy . . . 11

3 Task 8 11
3.1 The Final Proposed system . . . 11
3.2 Implementing the system . . . 12
3.3 Predicted performance . . . 14
3.4 Epiphany . . . 14

4 Maxeler: A different approach 15

5 Conclusions 15

1 Task 6: Add Hardware Floating point Units

Task 6 focuses on understanding the implementation of custom instruction hardware blocks to accelerate the execution time of the function through a custom hardware floating point unit. Once the work flow of custom instructions was understood a great potential to accelerate software through hardware was created and demonstrated. However the system created has certain limitations due to the interface of custom instructions with hardware blocks.

1.1 Setting up custom instructions

In order to access hardware external to the NIOS II processor, Qsys provides configuration options to interface such external hardware. Since these instructions have to be considered in the architecture of the processor there are certain limitations. These limitations are given by the necessity of defining the instruction type as combinational, multi-cycle, or extended, and in each type a given interface of inputs and outputs are defined. Moreover the hardware for an instruction implementing each instruction type must follow specific timing requirements.

In the case of the first custom instruction implementation, a single cycle integer multiplication, meeting the timing requirements was quite simple. The only requirement was for the combinational circuit to meet the timing requirements from new data available in the dataa and datab ports to the result port. This generated a macro in the system header of the type:

```
int __builtin_custom_inii (int n, int dataa, int datab);
```
(1)

There was also an additional macro to define n as the appropriate opcode that was determined in Qsys. The signature of the function to use was therefore `ALT_CI_FIRST_MUL_0(A,B)` which was successfully tried in our main function.

In order to implement a hardware floating point unit a different approach was taken. While each instruction in the hardware floating point could have been added individually as a custom instruction, a single hardware unit approach was taken. This was decided in order to more similarly reproduce the hardware floating point unit available in Qsys. Selecting such an implementation, not only allowed us to better understand the concept of a floating point unit, but also allowed us to write more maintainable and efficient code. Using a single custom instruction reduces the need of multiple connections to the custom instruction ports in the Nios. It also allows for the reuse of hardware within the floating point unit. For example, both the add and subtract instructions are implemented by the same instance of `ALTFP_ADD_SUB`. This block approach would have also allowed for more resource sharing and control over the hardware if we had not used such library functions. However designing hardware for floating point operations is complicated and as seen in lectures, library implementations are very optimized.

Since the different floating point operations from library implementations have different minimum latencies, the block implemented an extended custom instruction, as they allow for multiple number of cycles required for completion. This allowed for a 3rd input n to be used as an opcode to determine the operation of the hardware block. The library implementations are all pipelined and allowed to define the latency of the operation from a given minimum (for example 5 for fp multiply). This could be used in a pipelined system which uses several floating point operations in parallel to further operate on the results in the appropriate cycle. In this case however the overall hardware block can not be pipelined because the processor stalls until the done signal is asserted, and therefore the minimum latencies were chosen. This prevents us from supplying new inputs to the hardware block until an output is received, hence the aim was to minimize the latency from input to output. The done signal was therefore asserted after the appropriate number of cycles given the operation determined by n. The only problem encountered was that the data has to be available in the result port during the rising edge when the done signal is asserted. The done signal was therefore asserted half a cycle after the floating point operation was completed by the library module. Then in the next rising edge of the clock the done signal will be asserted and the output of the floating point operation will have propagated and will be available in the result port.

1.2 Our hardware floating point unit

The floating point hardware unit that is implemented was a combination of standard floating point units and custom ones. The floating point hardware unit ('`FP_HW.V`') contains the following three units:

- floating point addition and subtraction (standard unit)
- floating point multiplication (standard unit)
- floating point to fixed point conversion (custom unit)

The addition and subtraction block as well as the multiplication block were extracted from the normal floating point library implementations. This reduces the chance of introducing our own bugs, since this designs should be reliable. On the other hand, the standard floating point conversion unit was inadequate for the desired operation. This custom unit performed standard floating point conversion from a floating point number to a rounded integer. The desired operation however was to cast the floating point to an integer as seen in Task 5, and therefore not to round the floating point but to truncate it. Our own custom instruction performed this operation in a single cycle by shifting the mantissa by the exponent and retaining only the bits after the decimal point. Moreover since our inputs are constrained to the interval from 0 to 255 our unit did not consider negative inputs nor inputs with an exponent greater than the available 32 bits. Any other floating point conversion operations that occur utilize a standard floating point conversion unit, as this custom floating point conversion unit is specifically tailored only to this situation.

1.2.1 Performance Results

The results that will be presented are composed of two sets. One set is the replacement of only the operations within the target function with the custom floating point instruction. The other set also replaces the accumulation of each value with the floating point hardware. The reason for this was to illustrate the fact that any floating point operation done with the + operator instead of our custom instruction is still implemented in software. This was actually learned because we simply did not realize that we were accumulating the results with the += operator. This realization also allowed for a more efficient implementation in task 7.

The effect of implementing a hardware floating point unit is quite clear, as is show in Table 1

Impelmentation	No FP Unit	FP Unit (Without FP Accumulation)	FP Unit (With FP Accumulation)
Test 1 runtime (100 μs):	596	583	571
Test 2 runtime (100 μs):	28,951	28,699	28,132
Test 3 runtime (100 μs):	2,972,979	2,938,547	2,879,494

Table 1: Effect of implementing custom floating point hardware instructions

The improvement when not utilizing the floating point accumulation is 2.18%, 0.87% and 1.16% for Task 1, 2 and 3 respectively. Utilizing floating point hardware clearly already provides an improvement just for calculating the value of the function. When utilizing the floating point hardware to its full potential, the speed improvement became 4.19%, 2.83% and 3.14% for Task 1, 2 and 3 respectively. Clearly every addition of floating point instruction used provides a benefit regarding execution time. The improvement is still relatively limited in its effect. The reason for this is because the cosine operation is currently taking up a large amount of time for processing. Considering that the implementation of floating point instructions does not affect the speed of processing a cosine instruction (see next sub section), the degree of improvement is very limited. The impact of speeding up only part of the execution can be predicted by Amdahl's law as shown in Task 8. For this case the law is used as a tool to reason that most of the execution time is spent in the cosine and therefore acceleration efforts should be aimed to reduce this execution time. Task 7 therefore places a large focus on abstracting the entire process to hardware, including the cosine calculations.

1.2.2 Cache Size Optimization

As stated for previous tasks, the sole aim is not only to reduce runtime, but to produce the best $\frac{\text{Throughput}}{\text{FPGA Resource}}$ utilization. The data that has been presented so far has been for a cache size of 8 Kbytes. This is the remnant from Task 5, but should be reconsidered as the size of the executed instruction loop has been reduced, because of a shift of floating point operations from software to hardware. The more processes that are abstracted to hardware, the less instructions that the system needs to worry about, the smaller the cache can become.

Cache Size	8 (kbytes)	4 (kybtes)
Test 1 throughput (instr/second)	910.6830123	763.5829662
Test 2 throughput (instr/second)	906.7965306	761.7426618
Test 3 throughput (instr/second)	885.5757296	747.5132822

Table 2: Throughput for various Cache Sizes

As can be seen from Table 2, the throughput clearly increases as the cache size increases. Just like in Task 5, this indicates that more cache size allows for faster instruction access and far better throughput.

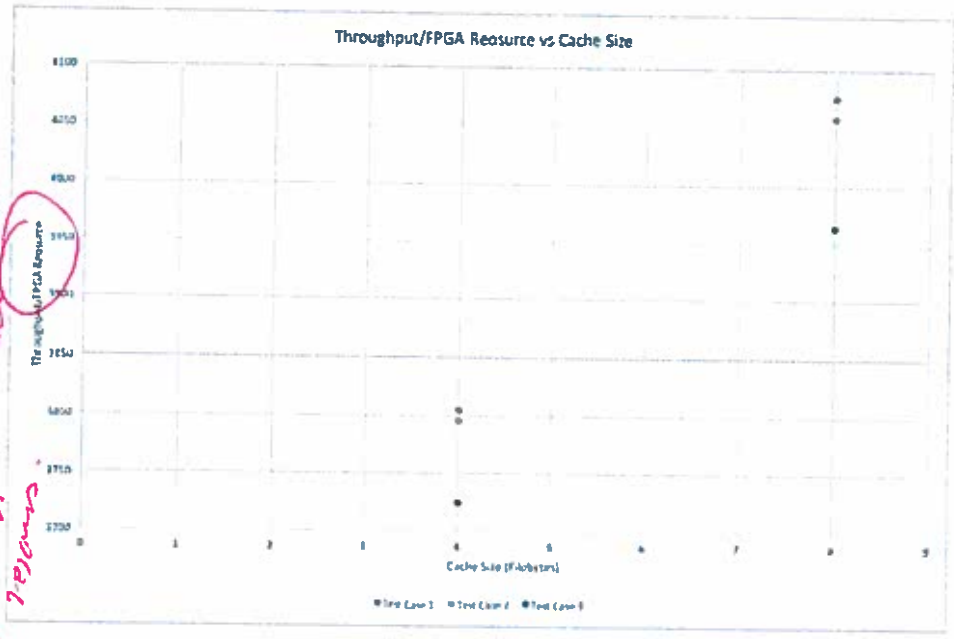


Figure 1: Throughput per FPGA Resource

Figure 1 demonstrates that not only does using 8 kilobytes of cache provide best throughput, it is also the most efficient use of resources. This provides the same conclusion as in Task 5. Having the same result as in Task 5 is not surprising, because even though the amount of instructions is decreased by providing a custom instruction, the reduction this provides is clearly not drastic enough to half the size of the cache. Only a very drastic reduction in instruction set size will allow for further cache size reduction.

1.2.3 Result Accuracy

The values produced for Task 6 are very similar to those in Task 5. The only difference is for Test 2, this set up proves to be slightly more accurate. The slight increase in accuracy is 0.0003%, which is an exceptionally small value, making the error improvement almost negligible.

Test Case	Absolute Error	Relative Error
1	0.027026888	0.0000467%
2	0.925036486	0.0012018%
3	198.0051449	0.0005348%

Table 3: Absolute and Relative Error Between Matlab and NIOSII Results

j

Having such incredibly similar results demonstrates that the implementation of floating point instructions does not trade in speed for accuracy, as the accuracy of the library functions matches that of the software implementations.

1.3 Extending the use of our hardware

The implementation of a custom hardware floating point unit allowed for the optimization of all arithmetic operations present within the direct calculations being performed. Even though this clearly provided runtime improvement more improvement could be achieved. In this setup only the operations we specifically mapped to our hardware were benefiting from it. All multiplications that occurred 'behind the scenes' (i.e. in the math.h library), were all still using software simulated floating point functions. Qsys allows the implementation of its own hardware floating point unit which can be used in eclipse as a normal multiplication operation, without the need of specific instructions. It was attempted to provide a similar functionality to our hardware block, such that the calculations of the cosine also benefited from the newly introduced hardware.

The first idea was to map all the multiply operations by overloading the multiplication operation. Research showed that this was not possible since such operator is not over-loadable. This promoted researching into alternative methods that would produce the same effects, even resorting to asking on stackoverflow (follow this

link [1]) to see if there were any alternatives to be considered. This produced varied alternatives, but not very conceivable considering the constraints placed on the project. One of the key points learned though was that the cosine calculations performed in the math library probably do not even use the '*' operator. This lead to exploring the finer details of the math.h library implementation of cos. The implementation that is present in the math.h library would not benefit from utilizing the overloaded operator, as all instructions are written in assembly, as can be seen in Figure 2

```
movd    %xmm0, %eax             /* Bits of x */
movaps  %xmm0, %xmm7            /* Copy of x */
cvtss2sd %xmm0, %xmm0           /* DP x */
movss   L(SP_ABS_MASK)(%rip), %xmm3
andl    $0x7fffffff, %eax       /* |x| */

cmpl    $0x3f490fdb, %eax       /* |x|<Pi/4?  */
jb      L(arg_less_pio4)

/* Here if |x|>=Pi/4 */
andps   %xmm7, %xmm3            /* SP |x| */
andpd   L(DP_ABS_MASK)(%rip), %xmm0     /* DP |x| */
movss   L(SP_INVPIO4)(%rip), %xmm2      /* SP 1/(Pi/4) */

cmpl    $0x40e231d6, %eax       /* |x|<9*Pi/4?  */
jae     L(large_args)

/* Here if Pi/4<=|x|<9*Pi/4 */
mulss   %xmm3, %xmm2            /* SP |x|/(Pi/4) */
cvttss2si %xmm2, %eax           /* k, number of Pi/4 in x */
lea     L(PIO4J)(%rip), %rsi
addl    $1, %eax                /* k+1 */
movl    $0x0e, %edx
andl    %eax, %edx              /* j = (k+1)&0x0e */
addl    $2, %eax                /* n */
subsd   (%rsi,%rdx,8), %xmm0    /* t = |x| - j * Pi/4 */
```

Figure 2: Diagram of Original Pipelined Function
https://sourceware.org/git/?p=glibc.git;a=blob;f=sysdeps/x86_64/fpu/s_cos.S;h=81c9757a01b129ab0f8e4ea3eadae15cf644aee;hb=HEAD

Implementing the Qsys specific floating point unit allows to recompile the newlib to support the floating point operations that were just created. This recompilation was initially not considered since it was believed that is was only applicable for floating point instructions of the qsys specific block. This was tried anyway with the newly introduced systems opcodes, hoping it would result in the same effect. This flags forced the recompilation of the newlib library, which is a lengthy process and did not work anyways.

Further effort was not placed into this since Task 7 focuses accelerating the cos function anyways.

[1] http://stackoverflow.com/questions/42421352/replacing-every-multiplication-with-a-custom-instruction/42421727#42421727

2 Task 7: Add Dedicated Hardware for f(x)

2.1 CORDIC Algorithm Implementation

Cosine calculation is often a very costly and the calculation can be greatly improved by implementing the CORDIC algorithm. The CORDIC algorithm provides a way of calculating trigonometric values using only addition, subtraction and shifts. These operations can be quickly performed in hardware with a low area cost.

There are many basic implementations of the the CORDIC algorithm online, such as the designs presented at: http://hdlexpress.com/Verilog/VT.html. This provided a very solid introduction into the algorithm and provided a basis for implementation, but was ultimately not the method that was used for implementation.

Most CORDIC implementations operate in the range between -90 to 90 degrees (as this is approximately the range CORDIC can account for). Any angle that is outside this range will require the initial value of x and y need to be altered. When generically calculating cosine, one must always find angle in the range $[0, 2\pi]$, which is performed by dividing *angle* by 2π. The necessity of having to normalize the angle is taken as an advantage in the implementation for this report by using the step to map any input to the $[0, \frac{\pi}{2}]$ range to also find the quadrant were this input lies. The CORDIC function implemented therefore only performed calculations in the range 0 to 90 degrees, and so the input angle must be found in range $[0, \frac{\pi}{2}]$ as:

$$\frac{\text{input angle}}{\frac{\pi}{2}} = \{\text{MSB's} : [1{:}0]\ \text{quadrant}\},\ \textit{remainder}\ \{[31{:}0]\ \text{angle in quadrant}\} \tag{2}$$

The MSB's of the division represent how many times 2π divides into the input and are irrelevant. The first 2 bits left of the decimal point represent the quadrant, since it is the number of times $\frac{\pi}{2}$ divides into the input after the multiples of 2π are removed. From the integer division as presented in Equation (2), the first two bits of the quotient provided the quadrant of the current angle. As this division has to occur anyways, it comes at not extra computational expense to utilize the quotient to determine the quadrant that the angle resides in. Depending on which quadrant the current angle is in, the initial values for x and y are altered accordingly to ensure the CORDIC algorithm does not need to internally change.

Furthermore, the degree of accuracy that was sought after was 32 bits for a fixed point decimal, since it was observed this maintained the accuracy of previous tasks. As stated in Lecture 7 of this DSD course, for k bit accuracy k steps of the CORDIC algorithm must be performed. As a result, the amount of iterations (displayed as `it_max` in Algorithm 1), has been set to 32 iterations to supply a sufficient degree of accuracy. The rest of the implementation follows the standard CORDIC algorithm.

When creating the CORDIC block, by utilizing the generate keyword in verilog, it was very easy to implement a bit-parallel unrolled CORDIC algorithm, allowing for a throughput of 1 cycle. The code for generating a bit-parallel unrolled setup can be seen in Algorithm 1:

Algorithm 1 fully pipelined bit-parallel unrolled CORDIC

```
genvar i;
generate
  for(i = 0; i < it_max; i = i + (pipeline_degree)) begin
    always @(posedge clock) begin
      ...
    end //always look
  end //for loop
endgenerate
```

Each statement within the 'for loop' that relates to i will form a separate pipelined stage. The amount of steps of the CORDIC algorithm that are performed within each stage of the pipeline is equal to the value of `pipeline_degree`. The 'TESTCORDIC.DO' file clearly demonstrates the pipelining that occurs which can be most clearly be seen through variable x, y or z.

The degree of unrolling can be altered to improve the latency of the entire CORDIC block.

Algorithm 2 Partially pipelined bit-parallel unrolled CORDIC

```
...
// pd = pipeline_degree
if (z[i][precision]) begin
  x[i+1] = x[i] + (y[i] >>> i);
  y[i+1] = y[i] - (x[i] >>> i);
  z[i+1] = z[i] + e[i];
end
else begin
  x[i+1] = x[i] - (y[i] >>> i);
  y[i+1] = y[i] + (x[i] >>> i);
  z[i+1] = z[i] - e[i];
end

if (z[i+1][precision]) begin
  x[i+2] = x[i+1] + (y[i+1] >>> (i+1));
  y[i+2] = y[i+1] - (x[i+1] >>> (i+1));
  z[i+2] = z[i+1] + e[i+1];
end
else begin
  x[i+2] = x[i+1] - (y[i+1] >>> (i+1));
  y[i+2] = y[i+1] + (x[i+1] >>> (i+1));
  z[i+2] = z[i+1] - e[i+1];
end
...
if (z[i+(pd-1)][precision]) begin
  x[i+pd] <= x[i+(pd-1)] + (y[i+(pd-1)] >>> (i+(pd-1)));
  y[i+pd] <= y[i+(pd-1)] - (x[i+(pd-1)] >>> (i+(pd-1)));
  z[i+pd] <= z[i+(pd-1)] + e[i+(pd-1)];
end
else begin
  x[i+pd] <= x[i+(pd-1)] - (y[i+(pd-1)] >>> (i+(pd-1)));
  y[i+pd] <= y[i+(pd-1)] + (x[i+(pd-1)] >>> (i+(pd-1)));
  z[i+pd] <= z[i+(pd-1)] - e[i+(pd-1)];
end
...
```

Algorithm 2 demonstrates how various pipeline stage sizes can be implemented. This is to be placed inside the '...' in Algorithm 1. The variable pd refers to the same variable `pipeline_degree` from Algorithm 1. It is useful to note that the throughput of the CORDIC block will always be 1 cycle, regardless of which implementation of Algorithm 2 is implemented. The only thing that Algorithm 2 affects is the overall latency of the CORDIC block. As can be seen, only blocking assignments are used except for the last `if` statement. The use of a non-blocking assignment at this point indicates that a pipeline stage has ended and that the next set of values belong to the next pipeline stage. The more operations that occur within one pipeline stage, the lower the latency. The limitation is that at one point, the amount of operations needed to complete a stage creates a critical path with timing constrains not satisfied and therefore incorrect results. The effects of attempting to place too many steps per cycle can be seen for a latency of 2 cycles in the results for Task 7.

2.2 Implementing the full instruction

With the implementation of CORDIC and the hardware floating point unit all the parts of the expression to implement could be accelerated. However this would not take advantage of the potential for parallelism and pipelining. As already mentioned pipelining custom instructions is not possible due to the forced stall of the processor until instruction completion. The approach was however to implement the instruction in a pipelined function for use in the next stage.

$$x(0.5 + x * cos(floor(x/4) - 32)) = x(0.5 + x * cos((int)x >> 2) - 32)) \quad (3)$$

In order to implement the function, the optimized equation 3 was used since it required less operations. The equivalence of the right hand side of the equation was already explored in the previous report and can be easily implemented given the floating point truncating module developed in task 6. The inner part of the cosine function was therefore implemented in a single block since its simplicity allowed its implementation in a single cycle. This comes from the fact that fixed shifts are hardware inexpensive and that the subtraction is a constant integer subtraction. From this equation the pipeline shown in figure

3 was developed.

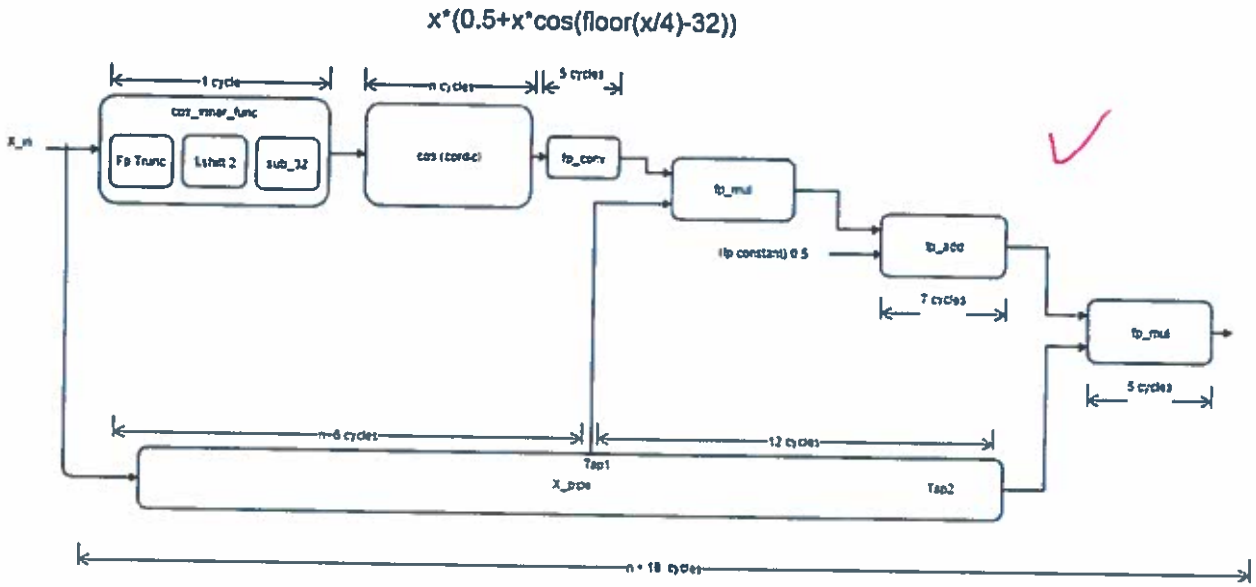


Figure 3: Diagram of the pipeline for the optimized function

The new component that allows for a pipelined implementation is the **x_pipe**. This ensures that the correct value of x is used at every stage. The use of library component LPM_SHIFTREG was explored since it will be optimized for the available hardware. An option for a memory based shift register is available since using FPGA slices only for the registers and not using the LUT can be a waste of resources. This memory approach is however more beneficial for larger shift register sizes and requires that the taps are evenly spaced, as can be the case in image processing applications. The LUT based version was therefore explored. From what we learned from the MegaWizard PlugIn manager, the LPM_SHIFTREG can only support 1 bit word inputs. The generated verilog was modified to create taps at parametrized delays and a generated for loop was used to accommodate the 32-bit word. The reported usage for 1 bit shift is 1 LE per delay length required. However various issues with this implementation were encountered and a custom verilog block was developed. This allowed a more maintainable implementation and the reported usage utilization was less than the expected utilization in 32 shift registers, since there is logic involved that can be mapped into the LUT's and therefore better utilize LE's.

A closer inspection of the first pipeline proposed revealed that the optimization into Equation 3 reduced the potential for parallelism in the pipeline. The new proposed pipeline shown in Figure 4 uses the commutative property of multiplication to reduce the latency by 5 cycles at the expense of one additional floating point multiplication module. Given that for this task the pipelining of inputs is not possible and latency is essential this was deemed a beneficial performance for area trade-off.

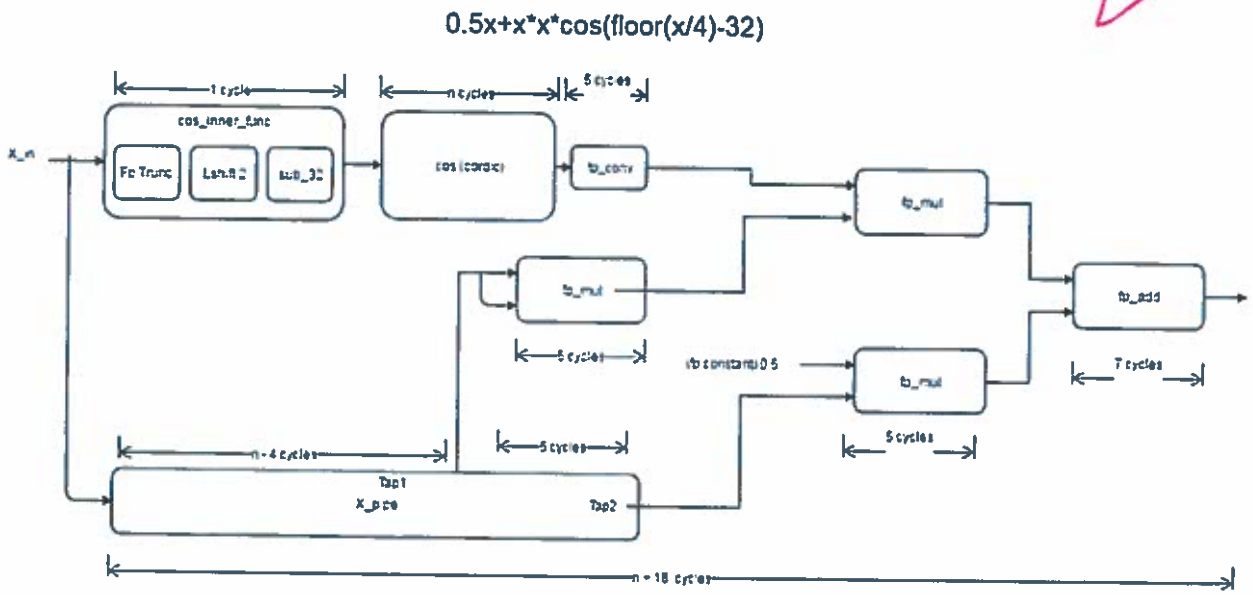


Figure 4: Diagram of the pipeline implementing the original function

Given that pipelining [illegible] in this setup the resource could had been minimized by varying the Latency

A

Latency (cycles):	Figure 4	Figure 5
Task 1 runtime (100 μs):	10	3
Task 2 runtime (100 μs):	470	126
Task 3 runtime (100 μs):	46732	12634

Table 4: Result for various designs

This solution was implemented and tested for the three cases to yield a speed increase shown in Table 4. The substantial decrease in execution time is obtained thanks to the hardware implementation of the cos function as well as the exploitation of parallelism in the pipeline. Moreover, as discussed in Task 6 an extra cycle is required for propagation from data generated synchronously in the hardware and the result port. The results for each input however were added without hardware support. The floating point hardware from Task 6 or a single floating point addition instruction could have been implemented. Again including this operation in the custom hardware circuit allows for more parallelism. Therefore the last and most efficient design in Figure 5 takes as inputs the previous accumulated value and the new input to generate the new accumulated value. In this design the fact that the inputs are not pipelined is used to remove the delay block and obtain the most performance and area efficient design for Task 7. The results for this final design can also been observed in 4. In such table the benefits of accumulating the result through hardware addition can be appreciated.

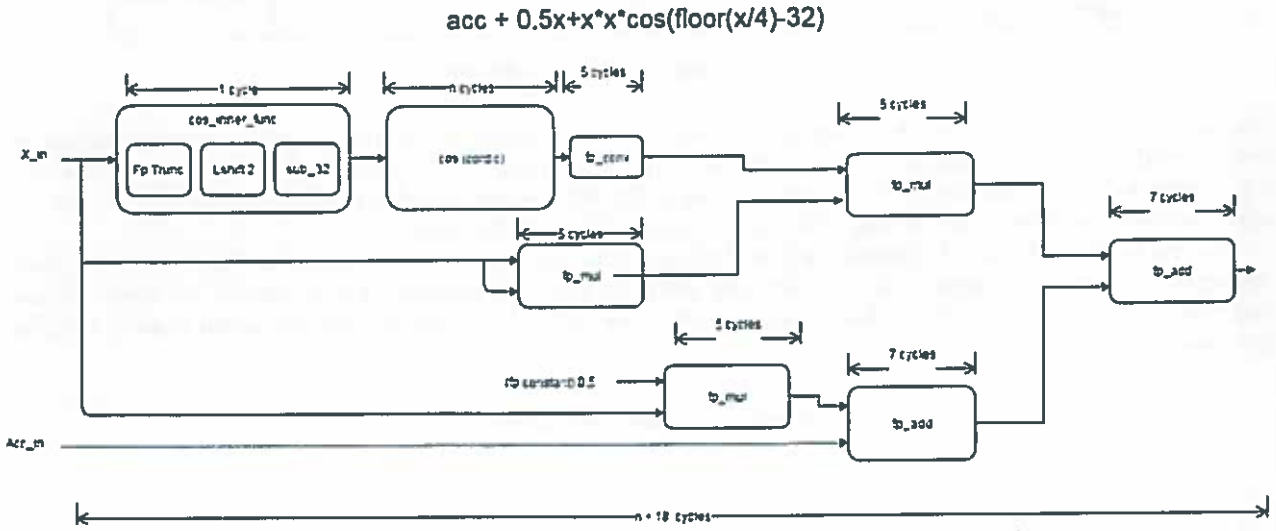


Figure 5: Diagram of Pipelined Accumulation Function

2.3 Task 7 Results

2.3.1 Latency Reduction Effects

Since the processor blocks to wait for the output of every input, the pipeline's utilization is very low and the pipeline's latency has a great impact. As a result, varying the various degrees of pipelining of the CORDIC algorithm have been tested and presented in Table 5:

CORDIC Latency (cycles):	32	16	8	4	2
Task 1 runtime (100 μs):	3	3	3	3	3
Task 2 runtime (100 μs):	126	117	112	110	109
Task 3 runtime (100 μs):	12634	11724	11189	11001	10849

Table 5: Effect of implementing custom floating point hardware instructions

As a note: $latency = \frac{32}{\text{stops per cycle}}$ since there are 32 iterations to be performed for every CORDIC input. Table 5 portrays the progressive decrease in runtime of the system overall as the latency of CORDIC and therefore of the whole instruction decreases. As Task 1 performs any cosine calculation the least amount of times (only 52), in terms of absolute time improvement it will clearly benefit a lot less from the the new implementation of the CORDIC algorithm. Task 2 with 2551 cosine calculations and Task 3 with 255001 cosine calculations will benefit to a far greater degree, as the effect of decreasing the latency of this single operation is exacerbated. The implementation of the CORDIC algorithm at its best has resulted in an almost 200 ms runtime reduction for Task 3. When considering the relative improvement that occurs, all three tasks are affected equally. It may not

seem that way for Task 1, but this is due to the fact that the relative improvement translates to an absolute improvement that is smaller than our smallest measurable time difference.

For the following task this optimization becomes irrelevant since th latency is less important in a pipeplined system. Then the most important factor to consider is to ensure that the CORDIC algorithm is not the critical path within the pipeline. Normally the clock frequency will be adapted to the critical path, so the smaller this is, the higher the maximum clock frequency. In the specifications the clock frequency is constrained to 50 MHz. In the current setup, having a latency of 2 cycles proved to be too ambitions and lead to incorrect results and therefore a pipeline line with a latency of 4 cycles will be used since it was the minimum achievable.

2.3.2 Cache Size Optimization

Cache Size	8 (kbytes)	4 (kybtes)	2 (kbytes)	1 (kybtes)	512 (bytes)
Test 1 throughput (instr/second)	173333.3333	173333.3333	173333.3333	173333.3333	173333.3333
Test 2 throughput (instr/second)	231909.0909	231909.0909	210826.4463	210826.4463	202460.3175
Test 3 throughput (instr/second)	231798.0184	231819.0909	210327.4497	210327.4497	203414.9649

Table 6: Throughput for various Cache Sizes

The size of the instruction set is yet again decreased in Task 7 compared to Task 6, as the entire function is transformed into a single instruction. Looking at Table 6, it is clear that using 8 kilobytes still provides the highest throughput of test cases per execution second. The throughput is entirely unaffected for Test 1. This is again because the speed improvement can not be measured by our tick speed.

The trade-off to consider is between area and throughput, which can be in the graph in Figure 6 which plots throughput per resource against the different test cases. Since the throughput change can not be measured for test case 1 it would appear that a 512 byte cache would be preferable. This explains the linear pattern seen in the data for Test Case 1.

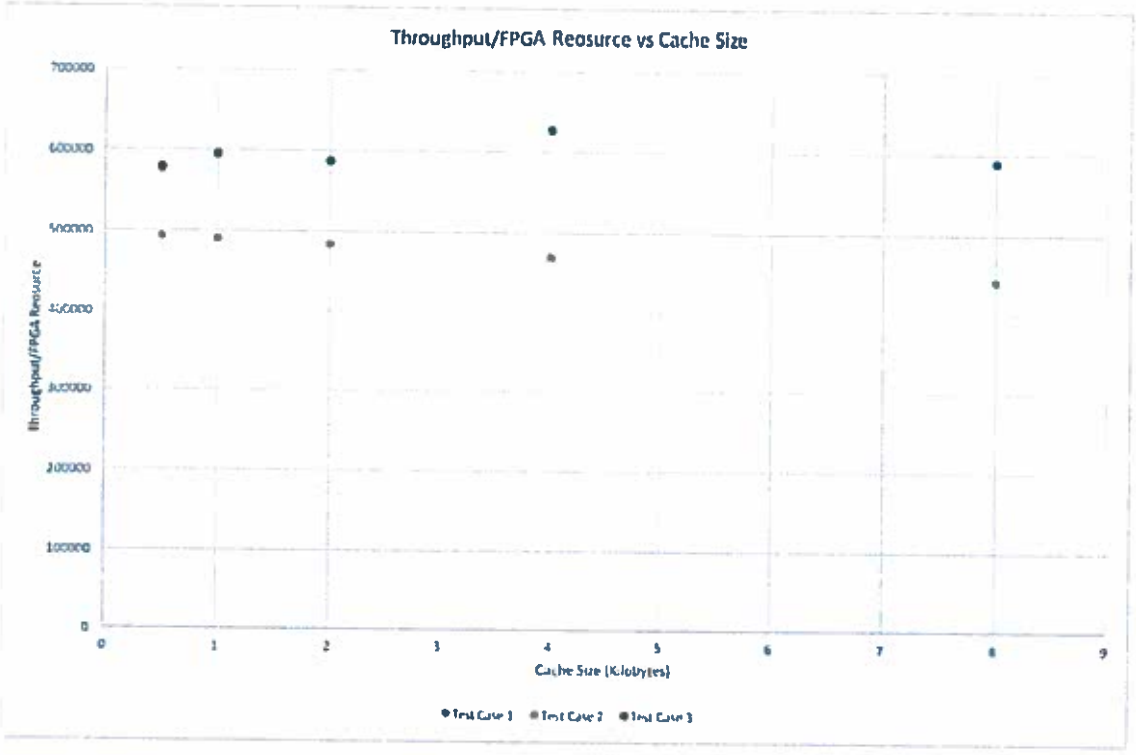


Figure 6: Throughput per FPGA Resource

For Test Cases 2 and 3 the data gives more valuable information regarding which cache to choose. As has been discussed before, the reduction in instruction set size has resulted in the ability to load the entire instruction set into a smaller cache. Table 6 demonstrated that a cache of sizes 8 and 4 kilobytes had very comparable

throughput. It therefore comes to no surprise that the Throughput per FPGA Resource for a 4 kilobyte cache size is notably better, as it in no way compromises speed but has half the the resource count. This was therefore the cache size used.

2.3.3 Result Accuracy

The values produced for Task 7 are identical to those in Task 6 and are therefore only presented here.

Test Case	Absolute Error	Relative Error
1	0.027026888	0.0000467%
2	0.925036486	0.0012018%
3	198.0051449	0.0005348%

Table 7: Absolute and Relative Error Between Matlab and NIOSII Results

Table 7 shows that the implementation of the CORDIC algorithm with the full instruction implemented in hardware is clearly no affecting the accuracy of the result, as it is identical to the values produced in Task 6. This is demonstrates the degree of accuracy that is used for the CORDIC algorithm is plenty to provide the same quality of results as that is expected from using the standard calculation techniques.

3 Task 8

3.1 The Final Proposed system

In order to make full use of the designed pipelined which can have a throughput of one output per cycle, it must be provided with one input per cycle. To prevent the processor from stalling every time it provides an input a FIFO was introduced as an input buffer. A new custom instruction allows loading the vector into the the buffer. Then two separate instructions are used to signal the pipeline to start using the values from the buffer and to block until the last value has been processed. In order to know when the last value has been processed, the pipeline expects to find a NaN floating point input after the last valid input. It will then wait until the last input has been processed, that is the pipelined has been emptied, to signal the end to the processor and supply the correct final output. A diagram of such input can be observed in figure 7.

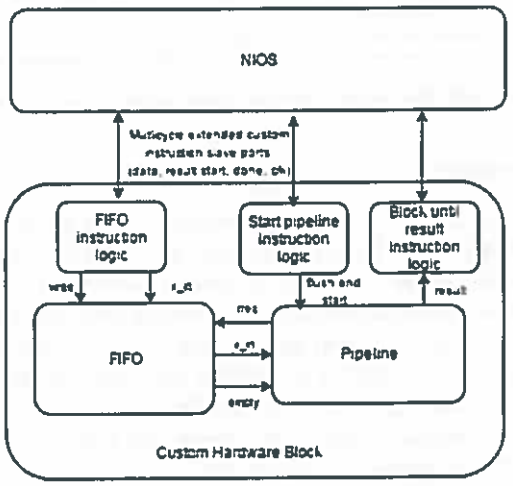


Figure 7: Diagram of the final system

In order to implement such system, an accumulated sum of all values has to be maintained. This posed two challenges. The first was to deal with the cycles where the buffer is empty so they do not corrupt the accumulated value. This was resolved by introducing a shift register to delay the bit holding the empty status of the buffer until the corresponding result was computed. Then a multiplexer can be used to just add 0 to the accumulated sum if the incoming value was not generated with valid buffer contents. This can be observed in the diagram of our final pipeline in figure 8. The second problem originated from having an additional floating point addition unit which had its output fed back as an input. Since the latency of the floating point addition unit is 7 cycles the current result of the accumulator is not ready. This means that you will get 7 partial sums which you should sum together after the last value has been processed. This solution would still maintain the 1 cycle throughput of the pipeline and would simply add 49 cycles of additional latency for the whole vector calculation latency. A

better alternative is to use specialized floating point accumulator hardware, since as described by [de Dinechin] and [de Dinechin, 2008], this hardware can be optimized for FPGA's.

The library block ALTERA_FP_ACC_CUSTOM was therefore chosen to implement the accumulation. Given the desired clock frequency, this IP block can be implemented with a 2 cycle latency. The implementation of the block also required certain parameters to be chosen carefully. The MSBA and LSBA parameters define the weights of the MSB and LSB respectively. For the case of the MSB this was straightforward, since based on the test cases the biggest expected result is $3.70225 * 10^7$. The weight of the MSB can therefore be found as $\lceil \log_2(3.70225 * 10^7) \rceil = 26$. The precision of the LSB can be defined as the precision per output value that we would like to keep. This would require an error analysis to determine the effect of this precision cap on the overall output error. Alternatively the document provides a formula to determine the precision on the LSB using the number of added inputs and the desired output precision as $LSBA = (-\textit{fixed point fraction precision} - \lceil \log_2(\textit{elements added}) \rceil)$. For example if $0.5 = 2^{-1}$ digits of precision are desired at the output of the summation of 10 terms then $LSBA = -1 - \lceil \log_2(10) \rceil = -5$. The problem is then the fact that our design has to accommodate for a variety of addition numbers and precisions. For the first test case the result is incorrect by an error smaller than 0.5 and only 52 elements were added. On the other hand the last test case has an error several orders of magnitude greater when adding 255001 terms. It was therefore decided to accommodate for the worst case in each of the two values. Choosing a precision at the output of 2^{-2} and having a maximum of 255,001 terms yields an LSBA of -20.

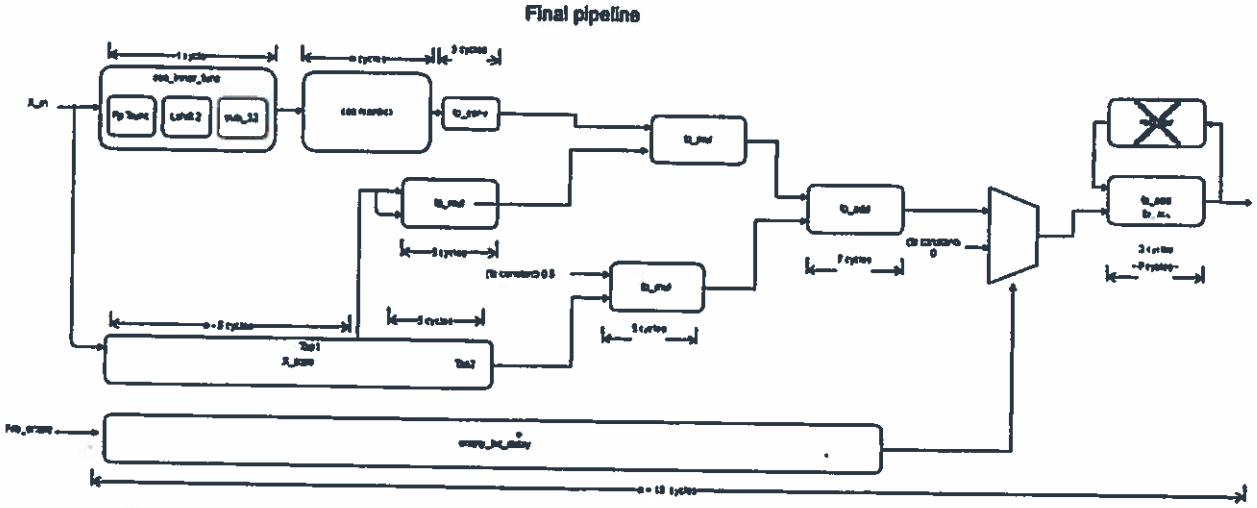


Figure 8: Diagram of the final pipeline with the necessary output accumulation and value flushing if created with an empty fifo

3.2 Implementing the system

All of the described system was implemented by adding the appropriate logic to the previous pipeline Verilog file. This additional logic controls inputing values from the Nios to the FIFO by dealing with the custom instruction signals. A multi-cycle instruction was chosen in order to have a start signal to use as a write request for the buffer. The combinational instruction does not produce a start signal and since the dataa and datab values are undefined when not running an instruction this can not be used to determined when to load an incoming value into the buffer. Similarly the instruction to start the pipeline also implements a multicycle instrution in order to generate a start signal which makes the pipeline start reading from the FIFO. This signal is latched until a value of NaN is read from the FIFO, which is used to represent the end of the inputs. Then a counter is used to determine when this last value has propagated through the pipeline. When the final result is ready the done signal of the end instruction is asserted. The end instruction and the start pipeline instruction are separate in order to allow the pipeline to run while the Nios is still loading values into the FIFO. Once the Nios has loaded the last value it can call the end instruction to block execution until a result is ready.

The problems started when simulating the whole system. The Modelsim version available to us does not support simulating a combination of files in multiple languages. Up until this point everything had been developed in Verilog, but the Altera floating point accumulator is only available as VHDL. Since developing a functioning system without simulation significantly increases the development time it was decided to implement our own floating point accumulator to be able to simulate the whole system. This accumulator consisted of library converters from floating point to fixed point and vice-versa, as well as an accumulating fixed point adder. This solves the issue with the latency of the floating point addition by considerably deepening the pipeline length (since the accumulating function has a latency of 12 cycles). The aim was to implement this suboptimal system first since we could simulate it and then to change the accumulator to the more optimal library block. We were able to simulate this system as seen in figure 9 and 10. In these figures it can be observed how the FIFO, pipeline

start and pipeline end instruction logic works. The done signals are generated at the appropriate time. This is especially important in the case of the end instruction. The simulation of figure 9 shows how the NaN input is supplied to trigger the flushing signal and related counter and the simulation of figure 10 shows how the done signal is not asserted until the flushing has ended.

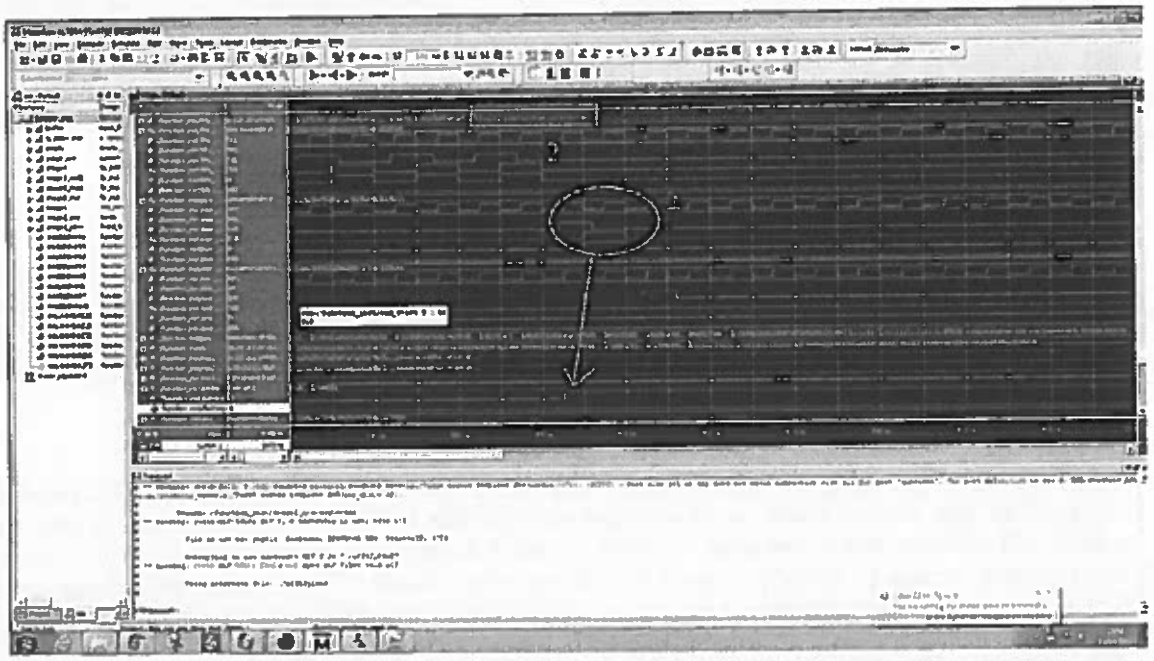


Figure 9: Initial part of the simulation of the final system were the behavior of the fifo placing instruction and of the pipeline starting instruction can be observed. The start and done signals of each instruction can be observed to be set correctly. Label 1 shows how the logic of the pipeline start instruction sets the running bit of the pipeline. label 2 shows the value NaN which is 0x7fffffff loaded to the fifo to signal the end of the vector.

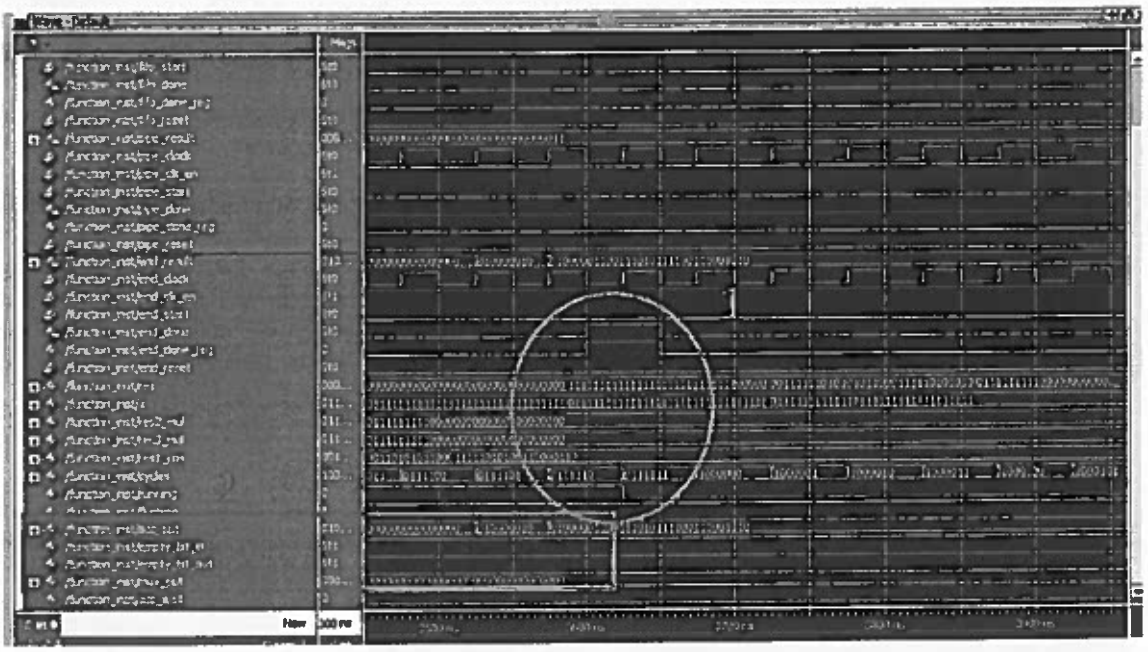


Figure 10: Final part of the final system simulation where the behavior of the end instruction can be observed. Label 1 show how the done signal of the instruction is asserted after the a certain number of cycles have elapsed with the flushing signal high. Label 2 shows how the mux_out, which feeds the accumulator is stuck at 0 since the values which are being processed are not valid any more

The 3 custom instructions were then carefully set up in the Nios system editor, and the appropriate functions were generated in the system header. However we then encountered several fatal issues. The first bug consisted in any output from the code in the form of `cout` or `printf` to not appear after 2 calls to any of these functions. This was tested by removing the working calls and finding that the next call was invoked. It is a hard bug to explain and we did not find its cause. Starting a new project fixed it, but it was a waste of time.

The bigger issue was the unexpected behavior of our custom instructions. While the functions to place data in the FIFO and to start the pipeline returned as expected, the end instruction, which is meant to block until the pipeline signaled the done port, returned unexpectedly. That is it was returning even if our hardware block did not generate a done signal. As confirmed by simulation our hardware does not provide a done signal unless it has received a NaN input. Even without supplying such signal the end instruction returns, as the execution of our code continues.

A slight improvement in this error was obtained by changing the system from having 3 custom instructions to a single custom instruction of type extended with an n input. The n input then defines the behaviour of the instruction to be either loading a FIFO value, starting the pipeline or waiting for the final result. Implementing this resulted in the end instruction now blocking execution.

3.3 Predicted performance

Since we were unable to fix the custom instructions to obtain the results from our pipeline we were unable to obtain the performance results for our final system. We can however provide an estimate based on theory. Based on the speedup from changing the latency of our pipeline in task 7, we can use Amdahl's law to derive the time spent by the processor on calling a custom instruction for each of the vector inputs. Amdahl's law states that:

$$S_{latency} = \frac{1}{(1-p) + \frac{p}{s}} \tag{4}$$

In the equation $S_{latency}$ is the speed up of the latency for a fixed load task, p is the proportion of the workload that can be sped up, and s is the speed up. In our case we would like to find out the proportion of time spent executing code and therefore not in the pipeline. Since in task 7 changing the depth of the CORDIC hardware from 32 to 16 reduced the latency from 50 cycles to 34, it is a speed up of 1.47. The overall speed up measured was from 1.2634 seconds to 1.1724 and therefore $S_{latency} = 1.076$. Substituting values we can obtain $p = 0.22$. This means only 22% of the time is spent waiting for results to propagate through the pipeline. In the new proposed system, there are still an equal number of instruction calls to load the vector into the fifo. The overhead of such instructions calls (including loading contents from memory and accessing the custom instructions) would remain the same. The difference would be that since the values are now accumulated in the the pipeline and computed in a pipelined fashion the isn't any more the overhead of waiting for the full latency of the pipeline for each input. We can therefore discard the 22% of the execution time and add the overhead of having to wait for flushing of the final result in the pipeline. Since the suboptimal pipeline with our own floating point accumulator has a latency of 62 cycles, running at a 50Mhz clock gives a negligible overhead of $1.24\mu s$. Then the expected latency of the last test case is of $1.2634 * 0.78 = 0.98s$.

Although we initially failed (keep reading) at measuring actual performance the speculated performance gives as a valuable insight to evaluate our system. Analysis showed that 88% of the execution time is spent in overhead including delivering the inputs to the pipeline. We can therefore predict that the bottleneck is in fact in memory accesses from Nios. This leads us to the conclusion that the utilization of our pipeline is very low. Since the throughput of loading values to the FIFO is lower than the throughput of our pipeline which consumes them, the FIFO spends most of the time empty and the pipeline computes values that are discarded. A better system would therefore have a higher throughput with memory. This can be obtained through a direct memory access controller (DMA) which is available through Qsys. This controller would allow burst reads from memory which would greatly increase the memory throughput and therefore the utilization of the pipeline, reducing the overall latency of each test case workload.

3.4 Epiphany

Having changed from 3 separate instructions to a single extended instruction seemed to show the desired execution except from the ending of the pipeline. Reviewing our system we concentrated on the causes of the pipeline end, which is ultimately triggered by the loading of the NaN. The final last error was found to be that an incorrect loading of the NaN value. A new custom instruction was done to load the NaN value `0x7fffffff` from its integer representation 2147483647. The system now behaved as expected with the logic of the last custom instruction ending once the pipeline had finished. The speed results were very close to those predicted through Amdahl's law, since Test Case 3 ran in 0.9455s. The difference from the predicted 0.98 comes from the fact that the workload is not exactly the same since the compiled code is different. We must however admit that the calculated values obtained from the system were completely incorrect. There is a bug we have not been able to fix since our simulations do show the correct results.

Test Case	Runtime (100 μs)
1	2
2	95
3	9455

Table 8: Final execution times for the test cases with the fully pipelined system of task 8

Since the performance analysis concluded that the memory accesses where the bottleneck and therefore the FIFO would be mostly empty, we resized it to a smaller size of 128 words to reduce the area usage. Moreover, the critical loop of execution in the software is smaller than in previous tasks, so the size of 4kBytes chosen for Task 7 was chosen for the reasons given there. This gave a final area usage of 36.57% as can be calculated from table 9.

Resource Type	Value	Percentage
Total LE	11,914	77%
Total Mult	25	22%
Memory Bits	52,052	10%
FPGA Resources	-	36.57%

Table 9: Final System Resource Breakdown

4 Maxeler: A different approach

The final result of this project has been a hardware accelerated implementation of a function using a soft-processor in the FPGA. As seen throughout the task, the soft-processor has the tendency to be the bottleneck. It therefore seems logical to move as much as possible of the execution into custom logic. As was concurrently learned in the Department of Computing course for Custom Computing, a different approach to a similar problem can be obtained through the Maxeler tools from Maxeler Technologies.

This company has derived the necessary tools to accelerate code ran in any desktop computer through a PCI interface with their own FPGA hardware. They provide a modified version of Java capable of describing hardware by generating graphs of the dataflow. These graphs are meant to operate on data streams and are then compiled as function calls with arrays as arguments for many different languages such as C++ or Python.

The potential of this arrangement is that the function calls stream the data given as array inputs to the FPGA and then implement the function described by the graph in a pipelined fashion. The tools even do such pipelining on their own, thus deciding the stages necessary to clock the design at the desired speed. This means that the code necessary to implement the function for this project could be developed in a few lines of MaxJ code. This would have allowed for a hardware accelerated version of our function in a really short design time.

Ideally we would have liked to try and implement our function using such tools in the available hardware in DoC. This however had an initial learning curve for setting everything up which was time infeasible. Moreover the results would be hard to compare since they ran in different hardware, at different clock speed and without the constraints given in the specs such as having to use the NIOS/s core

To summarize, Maxeler tools provide a different approach to hardware acceleration by providing certain abstraction from the hardware. The tools however still benefit from all the knowledge acquired in this project, since to obtain the most from them it is important to understand pipelining, identifying the bottlenecks of systems and exploring trade-offs in the design space.

5 Conclusions

Throughout the 8 tasks a system has been progressively built based on the previous tasks. Each subsequent design has improved performance measured as execution time while taking area into account. Every addition of hardware, from the embedded multipliers to a specialized pipeline, has result in an improvement from a software implementation in Nios. The Nios component of our system has therefore always been the bottleneck. This component does however provide a Hardware Abstaction layer which can allow the acceleration of software through FPGA hardware. This benefits from the potentiality of parallelism, pipelining and customization of digital designs while lacking the high design, test and manufacturing costs of ASICS. In conclusion, the work done has provided an insight into the vast design possibilities of digital system, exploring performance bottlenecks, optimizations and trade-offs to design the most suitable design for an application.

References

Bogdan Cret Octavian Tudoran Radu de Dinechin, Florent Pasca. *An FPGA-specific Approach to Floating-Point Accumulation and Sum-of-Products*, 2008.

Jérémie Cret Octavian de Dinechin, Florent Detrey. *When FGPAs are better at floating-point than microprocessors.*